

FRED TALKS

12th June 2026

CONTENTS

LLM Judges aren't the shortcut you think

DOUG TURNBULL · 1869 WORDS

Eight years of wanting, three months of building with AI

LALIT MAGANTI · 3910 WORDS

Giving LLMs a personality is just good engineering

SEANGOEDECKE.COM RSS FEED · 1062 WORDS

LLM Judges aren't the shortcut you think

DOUG TURNBULL · 03 NOV 2025 · [SOURCE](#)

Is “LLM as a judge” overhyped? After years of implementing LLM judges for clients, I find more of them recognize the limitations.

LLM judges fail for the same reason any evaluation method fails: teams lack good data+evaluation to begin with. LLM judges aren't the shortcut teams hope. Instead, they are themselves a thing that needs data and monitoring to get use out of.

Let me walk through the pitfalls of LLM as a judge (for search) that you'll encounter.

LLM as a judge

When I talk about LLM as a judge, my focus is search. I mean creating a [judgment list](#). For search that means labeling how relevant a document is for a query. How relevant is the product “garden trowel” for a `q=shovels`? Maybe this “garden trowel” is less relevant than “iron spade”?

In the ancient past, to improve the search, I would be limited to a tiny hand-labeled judgment list. Labeling by hand is expensive. Datasets drift out of date quickly. New use cases arise. So we constantly need humans to relabel.

These costs inspire teams to use LLMs to build judgments, They:

1. Start with a ground truth of human labels
2. Build a prompt to approximate that ground truth
3. Apply those prompts to go beyond the ground truth to new query / documents

If done well, we can sit at our laptop, tinker with relevance algorithms, and continue to tune without bother humans for non-stop relabeling.

Sounds promising. But teams often ignore many of the assumptions and nuance underlying the task of LLM labeling. Let's go through them, and we'll see at the end, maybe LLMs should play a different role in evaluation?

2. You could explain this in terms of “the stories we tell ourselves”. Many people (though not all) think that human identities are narratively constructed.

↩

3. I wrote about this last year in [Mecha-Hitler, Grok, and why it's so hard to give LLMs the right personality](#). A little nudge to change Grok's views on South African internal politics can cause it to start calling itself “Mecha-Hitler”.

↩

4. I have long believed that Claude “feels better” to use than ChatGPT because it has a more coherent persona (due mainly to Amanda Askell's work on its “soul”). My guess is that if you tried to make a “less human” version of Claude, it would become rapidly less capable.

↩

The base model is a kind of mysterious gestalt of its training data. If you feed it text, it will sometimes continue in that vein, or other times it will start outputting pure gibberish. It has no problem producing code with giant security flaws, or horribly-written English, or racist screeds - all of those things are represented in its training data, after all, and the base model does not judge. It simply outputs.

To build a *useful* AI model, you need to journey into the wild base model and stake out a region that is amenable to human interests: both ethically, in the sense that the model won't abuse its users, and practically, in the sense that it will produce correct outputs more often than incorrect ones. What this means in practice is that **you have to give the model a personality** during post-training¹.

Human beings are capable of almost any action at any time. But we only take a tiny subset of those actions, because that's the kind of people we are. I could throw my cup of coffee all over the wall right now, but I don't, because I'm not the kind of person who needlessly makes a mess². AI systems are the same. Claude could respond to my question with incoherent racist abuse - the base model is more than capable of those outputs - but it doesn't, because that's not the kind of "person" it is.

In other words, human-like personalities are not imposed on AI tools as some kind of marketing ploy or philosophical mistake. Those personalities are the medium via which the language model can become useful at all. This is why it's surprisingly tricky to "just" change a language model's personality or opinions: because you're navigating through the near-infinite manifold of the base model. You may be able to control which direction you go, but you can't control what you find there³.

When AI people talk about LLMs having personalities, or wanting things, or even having souls⁴, these are technical terms, like the "memory" of a computer or the "transmission" of a car. You simply cannot build a capable AI system that "just acts like a tool", because the model is trained on *humans* writing to and about other *humans*. You need to prime it with some kind of personality (ideally that of a useful, friendly assistant) so it can pull from the helpful parts of its training data instead of the horrible parts.

1. This is all pretty well understood in the AI space. Anthropic wrote a [recent paper](#) about it where they cite similar positions going all the way back to 2022. But for some reason it's not yet penetrated into communities that are more skeptical of AI.

↩

Problem one - LLM evals don't know what's engaging to users

Users' ever-evolving lizard brains drive conversions. And while optimizing for just lizard brains would itself be a nightmare, so would only focusing on an LLM's opinion.

LLMs do a better job approximating *human evaluations*. In search, we call these evaluations *explicit* judgments. They use human labelers' basic knowledge of the world to label a result as *topically relevant* to the query or not.

Topical relevance is only one kind of relevance. As an example, for a search of "articles about harry potter". The article is either factually about Harry Potter or its not. We might also consider other gradations:

- Is the article well written?
- Is it article from an authoritative source?
- Is the article about a tangential topic? Dumbledore vs Gandalf etc?

All of which relate to topicality, authority, and knowledge. An LLM excels at this — LLMs live in a world of facts and knowledge.

But LLMs don't have limbic systems. So their evaluations come with limitations.

In real search applications, "relevance" goes beyond topicality. I worked at Reddit on search. A search for "harry potter" might mean anything from a memes, to drama about JK Rowling, to every kind of juicy controversy, spicy post, or oddity. People search for "cybertruck" not for reviews but to see silly videos.

Then there's the more mundane. I found at Shopify, users preferred plain or black clothing over flashy items based on their clicks. Unless you tell an LLM this (because you've done the analysis) an LLM won't intrinsically catch this.

Often the "topical" part of search is relatively easy. But engaging search, that captures these human oddities, must go beyond world-knowledge to at least get *some* of a user's lizard brain.

Problem two - the last 10% of disagreement is the important stuff

Many teams get excited when they immediately see 70-80% agreement with human labelers. With a bit of tuning, they can inch closer to 90%.

In my experience, that last bit of human-LLM disagreement matters. Those are the non-obvious cases. Search deals with a big hard negatives challenge. Those examples that deceptively seem relevant naive search algorithms (or labelers), but are, in fact irrelevant. That last 10-30% might be what more careful labelers would catch. Or represented in clickstream based labels.

For example on this furniture dataset, a LLM doesn't understand that a "bistro table" is meant for ***outdoor*** use in US furniture lingo. That's not obvious to an LLM: The LLM imagines I'm opening a restaurant.

Simple algorithms often capture that first 80-90%. It's the 10-20% you need great labels for. So pay attention not just to coverage. But for which use cases.

A 90% coverage might be good regression tests, it might not help you go beyond easier wins.

Problem three - sneaky overfitting

So far, I've mentioned a few strange cases the LLM judge might miss: a bistro table is meant for outdoor usage. Users preferring darker clothing items.

The savvy prompt engineer will want to fill the prompt with examples to help the LLM perform better on these cases.

But beware overfitting to these particular failure cases.

With every new rule, you push the model's meager attention away from the general problem, and towards your exceptions. Layer example after example, and you'll find yourself with a brittle system focused excessively on specific use cases.

Overfitting with prompts can be sneaky. Playing whack-a-mole with your eval data can degrade generality. You can get perfect results on the eval set, only to expand to unseen queries, and find the gains no longer hold.

What I'd like to see more of from others is exactly what I've tried to do here: honest, detailed accounts of building real software with these tools; not weekend toys or one-off scripts but the kind of software that has to survive contact with users, bug reports, and your own changing mind.

Giving LLMs a personality is just good engineering

SEANGOEDECKE.COM RSS FEED · 03 MAR 2026 · [SOURCE](#)

AI skeptics often argue that current AI systems shouldn't be so human-like. The idea - most recently expressed in this [opinion piece](#) by Nathan Beacom - is that language models should explicitly be tools, like calculators or search engines. Although they *can* pretend to be people, they shouldn't, because it encourages users to overestimate AI capabilities and (at worst) slip into [AI psychosis](#). Here's a representative paragraph from the piece:

In sum, so much of the confusion around making AI moral comes from fuzzy thinking about the tools at hand. There is something that Anthropic could do to make its AI moral, something far more simple, elegant, and easy than what Askell is doing. Stop calling it by a human name, stop dressing it up like a person, and don't give it the functionality to simulate personal relationships, choices, thoughts, beliefs, opinions, and feelings that only persons really possess. Present and use it only for what it is: an extremely impressive statistical tool, and an imperfect one. If we all used the tool accordingly, a great deal of this moral trouble would be resolved.

So why do Claude and ChatGPT act like people? According to Beacom, AI labs have built human-like systems because AI lab engineers are trying to hoodwink users into emotionally investing in the models, or because they're delusional true believers in AI personhood, or some other foolish reason. This is wrong. AI systems are human-like because **that is the best way to build a capable AI system**.

Modern AI models - whether designed for chat, like OpenAI's GPT-5.2, or designed for long-running agentic work, like Claude Opus 4.6 - do not naturally emerge from their oceans of training data. Instead, when you train a model on raw data, you get a "base model", which is not very useful by itself. You cannot get it to write an email for you, or proofread your essay, or review your code.

When I was working on something where I didn't even know what I wanted, AI was somewhere between unhelpful and harmful. The architecture of the project was the clearest case: I spent weeks in the early days following AI down dead ends, exploring designs that felt productive in the moment but collapsed under scrutiny. In hindsight, I have to wonder if it would have been faster just thinking it through without AI in the loop at all.

But expertise alone isn't enough. Even when I understood a problem deeply, AI still struggled if the task had no objectively checkable answer³⁸. Implementation has a right answer, at least at a local level: the code compiles, the tests pass, the output matches what you asked for. Design doesn't. We're still arguing about OOP decades after it first took off.

Concretely, I found that designing the public API of `syntaqlite` was where this hit home the hardest. I spent several days in early March doing nothing but API refactoring, manually fixing things any experienced engineer would have instinctively avoided but AI made a total mess of. There's no test or objective metric for "is this API pleasant to use" and "will this API help users solve the problems they have" and that's exactly why the coding agents did *so badly* at it.

This takes me back to the days I was obsessed with physics and, specifically, relativity. The laws of physics look simple and Newtonian in any small local area, but zoom out and spacetime curves in ways you can't predict from the local picture alone. Code is the same: at the level of a function or a class, there's usually a clear right answer, and AI is excellent there. But architecture is what happens when all those local pieces interact, and you can't get good global behaviour by stitching together locally correct components.

Knowing where you are on these axes at any given moment is, I think, the core skill of working with AI effectively.

Eight years is a long time to carry a project in your head. Seeing these SQLite tools actually exist and function after only three months of work is a massive win, and I'm fully aware they wouldn't be here without AI.

But the process wasn't the clean, linear success story people usually post. I lost an entire month to vibe-coding. I fell into the trap of managing a codebase I didn't actually understand, and I paid for that with a total rewrite.

The takeaway for me is simple: AI is an incredible force multiplier for implementation, but it's a dangerous substitute for design. It's brilliant at giving you the right answer to a specific technical question, but it has no sense of history, taste, or how a human will actually feel using your API. If you rely on it for the "soul" of your software, you'll just end up hitting a wall faster than you ever have before.

To do a good job, you need holdout data you're not tuning against. A set of judgments you don't peak at. Because peaking might cause you to sneak a change into the prompt, making the holdout less useful as an independent validation step.

Add rules and exceptions carefully, you'll be surprised how easy it is to pull an LLM away from its generality.

Problem four - LLMs only evaluate the document it sees

LLMs only evaluate the document it sees. IE:

```
Query: outdoor seating
Product Name: Bistro Table
Product Description: Enjoy this lovely bistro table on your porch...
```

Is this enough information for evaluating the product? Is it what a user sees?

Suddenly, you decide to add new information about the product

```
Query: outdoor seating
Product Name: Bistro Table
Product Description: Enjoy this lovely bistro table on your porch...
Category: Outdoors
```

What are we evaluating truly?

- The products relevance?
- The usefulness of a set of product properties for determining relevance?
- How comprehensively you're describing products to an LLM?

Really it gets down to a classic problem in search. There's perceived relevance - the relevance of the product, as it appears in your product page. Then there's actual relevance - the detailed, thoughtful understanding of your product by a user. A lot depends not just on true relevance, but how it's presented to users.

Problem five - Your LLM judge can't be trusted on novel use cases

I assume the goal of LLM as a judge in to align to human labelers. But rarely do those judgments comprehensively represent every use case.

Search metrics with human evals don't go up-and-to-the-right. They seesaw as teams reactively label, solve, label, solve ad infinitum. In my experience it's rare to find a team build a true, comprehensive human labeled evals covering every use case. Even if we did, we'd need to account for evolving search use cases.

The flow usually goes like this: we want to solve a problem (search by product name). So we capture some judgments for that use case. Relevance starts low. We work a bit to improve this use case and it goes up. Yay!

But the boss comes to our desk and suggests we have a new problem to solve: search of furniture by the room (living room, etc). We gather more labels. We know we're bad at this use case. So search metrics go back down. Sad 😞

We solve that use case, it goes back up to where it was before. Repeat. Up and down. See-saw.

This presents a problem for an LLM judge. If our human labels only focus on a subset of use cases, **we're also aligning our LLM judges on a subset of use cases.**

Our LLM judge generalizes to just a piece of the problem. The LLM judge can't be trusted on a new use case until we measure it's abilities in that use case.

So if you seesaw its human labeling, you must seesaw the alignment of the LLM judge to the use cases we capture in human eval. As we adopt a new use case, we need more human labels. We need to retest the LLM judge. And so on.

The task of human labeling does not end with LLM Judges. In fact its even more crucial.

How to LLM as a judge

Here's how I use LLM judges:

- To extend labels on use cases I already measure decently to new queries (because I can trust alignment of the LLM on this use case)
- As a safeguard against regressions

Basically, I learned that the “normal rules” of software still apply in the AI age: if you don't have a fundamental foundation (clear architecture, well-defined boundaries) you'll be left eternally chasing bugs as they appear.

Something I kept coming back to was how little AI understood about the passage of time³⁶. It sees a codebase in a certain state but doesn't *feel* time the way humans do. I can tell you what it feels like to use an API, how it evolved over months or years, why certain decisions were made and later reversed.

The natural problem from this lack of understanding is that you either make the same mistakes you made in the past and have to relearn the lessons *or* you fall into new traps which were successfully avoided the first time, slowing you down in the long run. In my opinion, this is a similar problem to why losing a high-quality senior engineer hurts a team so much: they carry history and context that doesn't exist anywhere else and act as a guide for others around them.

In theory, you can try to preserve this context by keeping specs and docs up to date. But there's a reason we didn't do this before AI: capturing implicit design decisions exhaustively is incredibly expensive and time-consuming to write down. AI can help draft these docs, but because there's no way to automatically verify that it accurately captured what matters, a human still has to manually audit the result. And that's still time-consuming.

There's also the context pollution problem. You never know when a design note about API A will echo in API B. Consistency is a huge part of what makes codebases work, and for that you don't just need context about what you're working on right now but also about other things which were designed in a similar way. Deciding what's relevant requires exactly the kind of judgement that institutional knowledge provides in the first place.

Reflecting on the above, the pattern of when AI helped and when it hurt was fairly consistent.

When I was working on something I already understood deeply, AI was excellent. I could review its output instantly, catch mistakes before they landed and move at a pace I'd never have managed alone. The parser rule generation is the clearest example³⁷: I knew exactly what each rule should produce, so I could review AI's output within a minute or two and iterate fast.

When I was working on something I could describe but didn't yet know, AI was good but required more care. Learning Wadler-Lindig for the formatter was like this: I could articulate what I wanted, evaluate whether the output was heading in the right direction, and learn from what AI explained. But I had to stay engaged and couldn't just accept what it gave me.

Several times during the project, I lost my mental model of the codebase³¹. Not the overall architecture or how things fitted together. But the day-to-day details of what lived where, which functions called which, the small decisions that accumulate into a working system. When that happened, surprising issues would appear and I’d find myself at a total loss to understand what was going wrong. I hated that feeling.

The deeper problem was that losing touch created a communication breakdown³². When you don’t have the mental thread of what’s going on, it becomes impossible to communicate meaningfully with the agent. Every exchange gets longer and more verbose. Instead of “change FooClass to do X,” you end up saying “change the thing which does Bar to do X”. Then the agent has to figure out what Bar is, how that maps to FooClass, and sometimes it gets it wrong³³. It’s exactly the same complaint engineers have always had about managers who don’t understand the code asking for fanciful or impossible things. Except now you’ve become that manager.

The fix was deliberate: I made it a habit to read through the code immediately after it was implemented and actively engage to see “how would I have done this differently?”.

Of course, in some sense all of the above is also true of code I wrote a few months ago (hence the [sentiment that AI code is legacy code](#)), but AI makes the drift happen faster because you’re not building the same muscle memory that comes from originally typing it out.

There were some other problems I only discovered incrementally over the three months.

I found that AI made me procrastinate on key design decisions³⁴. Because refactoring was cheap, I could always say “I’ll deal with this later.” And because AI could refactor at the same industrial scale it generated code, the cost of deferring felt low. But it wasn’t: deferring decisions corroded my ability to think clearly because the codebase stayed confusing in the meantime. The vibe-coding month was the most extreme version of this. Yes, I understood the problem, but if I had been more disciplined about making hard design calls earlier, I could have converged on the right architecture much faster.

Tests created a similar false comfort³⁵. Having 500+ tests felt reassuring, and AI made it easy to generate more. But neither humans nor AI are creative enough to foresee every edge case you’ll hit in the future; there are several times in the vibe-coding phase where I’d come up with a test case and realise the design of some component was completely wrong and needed to be totally reworked. This was a significant contributor to my lack of trust and the decision to scrap everything and start from scratch.

I trust the LLM judge to flag a regression on the use cases my human labels cover. The LLM is aligned and measured against these. I don’t trust the LLM to discover new, unexpected ways users search. I look to the users for that, not LLMs.

Still many teams don’t even do decent monitoring of topical relevance. There are plenty of obviously broken search engines. The ones that would benefit from an LLM judge, likely would benefit from rounds of human labeling first needed before blindly trusting an LLM judge.

We already have automated data labeling at home kids

If we have enough labels to evaluate LLM as a judge on a use case, we have enough to train an ML model to recover those labels. We can then see a better place LLM’s fit into evaluation:

- LLMs focus on factual, topical parts of the problem. But miss other parts of the problem
- LLMs might see a subset of the content features
- Other non-LLM features might help us better evaluate relevance (embedding similarity, text matches, etc)
- Other ground-truths (besides human) might be a better definition of relevance (ie engagement based labels)

Instead of LLM as a Judge, **why not Learning to Rank as a judge?** We seem to have forgotten that ranking models provide value beyond directly deploying them to production. They can help guide more manual search tuning as well.

I’m becoming more of a fan of LLM-for-feature-generation over LLM as a judge. LLMs can cover the topical part of relevance. You can ask them “Which of these product names is more relevant to the query?”. The downstream, traditional ML model is agnostic to where the features come from. They could be an embedding similarity, LLM decision, lexical similarity, or more.

That’s much more my interest these days. When someone says “LLM as a judge” I nod and say yes let’s build that, but then go on and probably look into using LLM features heavily in a larger model. LLMs can be very useful for rapid feature development: they’re promptable and easy to play with.

If you’re interested in this approach, check out this [blog post](#) and the corresponding [Haystack talk](#). Also a talk from Shopify [choosing cross encoders for evaluation over LLMs](#)

Enjoy!

Enjoy softwareDoug in training course form!



I hope you join me at [Cheat at Search with LLMs](#) to learn how to apply LLMs to search applications. Check out [this post](#) for a sneak preview.

Eight years of wanting, three months of building with AI

LALIT MAGANTI · 12 APR 2026 · [SOURCE](#)

For eight years, I’ve wanted a high-quality set of devtools for working with SQLite. Given how important SQLite is to the industry¹, I’ve long been puzzled that no one has invested in building a really good developer experience for it².

A couple of weeks ago, after ~250 hours of effort over three months³ on evenings, weekends, and vacation days, I finally [released syntaqlite](#) (GitHub), fulfilling this long-held wish. And I believe the main reason this happened was because of AI coding agents⁴.

Of course, there’s no shortage of posts claiming that AI one-shot their project or pushing back and declaring that AI is all slop. I’m going to take a very different approach and, instead, systematically break down my experience building syntaqlite with AI, both where it helped *and* where it was detrimental.

This extended to entire domains I’d never worked in. I have deep C++ and Android performance expertise but had barely touched Rust tooling or editor extension APIs. With AI, it wasn’t a problem: the fundamentals are the same, the terminology is similar, and AI bridges the gap²⁴. The VS Code extension would have taken me a day or two of learning the API before I could even start. With AI, I had a working extension within an hour.

It was also invaluable for reacquainting myself with parts of the project I hadn’t looked at for a few days²⁵. I could control how deep to go: “tell me about this component” for a surface-level refresher, “give me a detailed linear walkthrough” for a deeper dive, “audit unsafe usages in this repo” to go hunting for problems. When you’re context switching a lot, you lose context fast. AI let me reacquire it on demand.

Beyond making the project exist at all, AI is also the reason it shipped as complete as it did. Every open source project has a long tail of features that are important but not critical: the things you know theoretically how to do but keep deprioritizing because the core work is more pressing. For syntaqlite, that list was long: editor extensions, Python bindings, a WASM playground, a docs site, packaging for multiple ecosystems²⁶. AI made these cheap enough that skipping them felt like the wrong trade-off.

It also freed up mental energy for UX²⁷. Instead of spending all my time on implementation, I could think about what a user’s first experience should feel like: what error messages would actually help them fix their SQL, how the formatter output should look by default, whether the CLI flags were intuitive. These are the things that separate a tool people try once from one they keep using, and AI gave me the headroom to care about them. Without AI, I would have built something much smaller, probably no editor extensions or docs site. AI didn’t just make the same project faster. It changed what the project was.

There’s an uncomfortable parallel between using AI coding tools and playing slot machines²⁸. You send a prompt, wait, and either get something great or something useless. I found myself up late at night wanting to do “just one more prompt,” constantly trying AI just to see what would happen even when I knew it probably wouldn’t work. The sunk cost fallacy kicked in too: I’d keep at it even in tasks it was clearly ill-suited for, telling myself “maybe if I phrase it differently this time.”

The tiredness feedback loop made it worse²⁹. When I had energy, I could write precise, well-scoped prompts and be genuinely productive. But when I was tired, my prompts became vague, the output got worse, and I’d try again, getting more tired in the process. In these cases, AI was probably slower than just implementing something myself, but it was too hard to break out of the loop³⁰.

better with concrete prototypes to play with and code to look at than endlessly thinking about designs in my head, and AI lets me get to that point at a pace I could not have dreamed about before. Once I took the first step, every step after that was so much easier.

AI turned out to be better than me at the act of writing code itself, assuming that code is obvious. If I can break a problem down to “write a function with this behaviour and parameters” or “write a class matching this interface,” AI will build it faster than I would and, crucially, in a style that might well be more intuitive to a future reader. It documents things I’d skip, lays out code consistently with the rest of the project, and sticks to what you might call the “standard dialect” of whatever language you’re working in¹⁹.

That standardness is a double-edged sword. For the vast majority of code in any project, standard is exactly what you want: predictable, readable, unsurprising. But every project has pieces that are its edge, the parts where the value comes from doing something non-obvious. For syntaqlite, that was the extraction pipeline and the parser architecture. AI’s instinct to normalize was actively harmful there, and those were the parts I had to design in depth and often resorted to just writing myself.

But here’s the flip side: the same speed that makes AI great at obvious code also makes it great at refactoring. If you’re using AI to generate code at industrial scale, you *have* to refactor constantly and continuously²⁰. If you don’t, things immediately get out of hand. This was the central lesson of the vibe-coding month: I didn’t refactor enough, the codebase became something I couldn’t reason about, and I had to throw it all away. In the rewrite, refactoring became the core of my workflow. After every large batch of generated code, I’d step back and ask “is this ugly?” Sometimes AI could clean it up. Other times there was a large-scale abstraction that AI couldn’t see but I could; I’d give it the direction and let it execute²¹. If you have taste, the cost of a wrong approach drops dramatically because you can restructure quickly²².

Of all the ways I used AI, research had by far the highest ratio of value delivered to time spent.

I’ve worked with interpreters and parsers before but I had never heard of Wadler-Lindig pretty printing²³. When I needed to build the formatter, AI gave me a concrete and actionable lesson from a point of view I could understand and pointed me to the papers to learn more. I could have found this myself eventually, but AI compressed what might have been a day or two of reading into a focused conversation where I could ask “but why does this work?” until I actually got it.

I’ll do this while contextualizing the project and my background so you can independently assess how generalizable this experience was. And whenever I make a claim, I’ll try to back it up with evidence from my project journal, coding transcripts, or commit history⁵.

In my work on [Perfetto](#), I maintain a SQLite-based language for querying performance traces called [PerfettoSQL](#). It’s basically the same as SQLite but with a few extensions to make the trace querying experience better. There are ~100K lines of PerfettoSQL internally in Google and it’s used by a wide range of teams.

Having a language which gets traction means your users also start expecting things like formatters, linters, and editor extensions. I’d hoped that we could adapt some SQLite tools from open source but the more I looked into it, the more disappointed I was. What I found either wasn’t reliable enough, fast enough⁶, or flexible enough to adapt to PerfettoSQL. There was clearly an opportunity to build something from scratch, but it was never the “most important thing we could work on”. We’ve been reluctantly making do with the tools out there but always wishing for better.

On the other hand, there *was* the option to do something in my spare time. I had built lots of open source projects in my teens⁷ but this had faded away during university when I felt that I just didn’t have the motivation anymore. Being a maintainer is much more than just “throwing the code out there” and seeing what happens. It’s triaging bugs, investigating crashes, writing documentation, building a community, and, most importantly, having a direction for the project.

But the itch of open source (specifically freedom to work on what I wanted while helping others) had never gone away. The SQLite devtools project was eternally in my mind as “something I’d like to work on”. But there was another reason why I kept putting it off: it sits at the intersection of being both hard *and* tedious.

What makes it hard and tedious

If I was going to invest my personal time working on this project, I didn’t want to build something that only helped Perfetto: I wanted to make it work for *any* SQLite user out there⁸. And this means parsing SQL *exactly* like SQLite.

The heart of any language-oriented devtool is the parser. This is responsible for turning the source code into a “parse tree” which acts as the central data structure anything else is built on top of. If your parser isn’t accurate, then your formatters and linters will inevitably inherit those inaccuracies; many of the tools I found suffered from having parsers which approximated the SQLite language rather than representing it precisely.

Unfortunately, unlike many other languages, SQLite has no formal specification describing how it should be parsed. It doesn't expose a stable API for its parser either. In fact, quite uniquely, in its implementation it doesn't even build a parse tree at all⁹! The only reasonable approach left in my opinion is to carefully extract the relevant parts of SQLite's source code and adapt it to build the parser I wanted¹⁰.

This means getting into the weeds of SQLite source code, a fiendishly difficult codebase to understand. The whole project is written in C in an incredibly dense style; I've spent days just understanding the virtual table API¹¹ and implementation. Trying to grasp the full parser stack was daunting.

There's also the fact that there are >400 rules in SQLite which capture the full surface area of its language. I'd have to specify in each of these "grammar rules" how that part of the syntax maps to the matching node in the parse tree. It's extremely repetitive work; each rule is similar to all the ones around it but also, by definition, different.

And it's not just the rules but also coming up with and writing tests to make sure it's correct, debugging if something is wrong, triaging and fixing the inevitable bugs people filed when I got something wrong...

For years, this was where the idea died. Too hard for a side project¹², too tedious to sustain motivation, too risky to invest months into something that might not work.

I've been using coding agents since early 2025 (Aider, Roo Code, then Claude Code since July) and they'd definitely been useful but never something I felt I could trust a serious project to. But towards the end of 2025, the models seemed to make a significant step forward in quality¹³. At the same time, I kept hitting problems in Perfetto which would have been trivially solved by having a reliable parser. Each workaround left the same thought in the back of my mind: maybe it's finally time to build it for real.

I got some space to think and reflect over Christmas and decided to really stress test the most maximalist version of AI: could I vibe-code the whole thing using just Claude Code on the Max plan (£200/month)?

Through most of January, I iterated, acting as semi-technical manager and delegating almost all the design and all the implementation to Claude. Functionally, I ended up in a reasonable place: a parser in C extracted from SQLite sources using a bunch of Python scripts, a formatter built on top, support for both the SQLite language and the PerfettoSQL extensions, all exposed in a web playground.

But when I reviewed the codebase in detail in late January, the downside was obvious: the codebase was complete spaghetti¹⁴. I didn't understand large parts of the Python source extraction pipeline, functions were scattered in random files without a clear shape, and a few files had grown to several thousand lines. It was *extremely* fragile; it solved the immediate problem *but* it was never going to cope with my larger vision, never mind integrating it into the Perfetto tools. The saving grace was that it had proved the approach was viable and generated more than 500 tests, many of which I felt I could reuse.

I decided to throw away everything and start from scratch while also switching most of the codebase to Rust¹⁵. I could see that C was going to make it difficult to build the higher level components like the validator and the language server implementation. And as a bonus, it would also let me use the same language for both the extraction and runtime instead of splitting it across C and Python.

More importantly, I completely changed my role in the project. I took ownership of all decisions¹⁶ and used it more as "autocomplete on steroids" inside a much tighter process: opinionated design upfront, reviewing every change thoroughly, fixing problems eagerly as I spotted them, and investing in scaffolding (like linting, validation, and non-trivial testing¹⁷) to check AI output automatically.

The core features came together through February and the final stretch (upstream test validation, editor extensions, packaging, docs) led to a 0.1 launch in mid-March.

But in my opinion, this timeline is the least interesting part of this story. What I really want to talk about is what wouldn't have happened without AI and also the toll it took on me as I used it.

AI is why this project exists, and why it's as complete as it is

I've written in the past about how one of my biggest weaknesses as a software engineer is my tendency to procrastinate when facing a big new project. Though I didn't realize it at the time, it could not have applied more perfectly to building syntaqlite.

AI basically let me put aside all my doubts on technical calls, my uncertainty of building the right thing and my reluctance to get started by giving me very concrete problems to work on. Instead of "I need to understand how SQLite's parsing works", it was "I need to get AI to suggest an approach for me so I can tear it up and build something better"¹⁸. I work so much